



THE UNIVERSITY OF
SYDNEY

The University of Sydney
School of Computer Science

Assignment1- Parallel Algorithm Design, Implementation, Testing and Report

This report submitted for the unit of study

COMP4426/5426

Parallel and Distributed Computing

APRIL 26, 2026

SID: 550219358

Name: EUNJIN CHOI

Table of Contents

0. Introduction..... 3

1.1 Problem Description 3

1.2 Key Requirements..... 3

1.3 Constraints 4

2. Parallel Algorithm Design and Workload Distribution..... 4

2.1 Algorithm Overview..... 4

2.2 Cost-Based Order Selection 4

2.3 Workload Distribution Strategy 5

2.4 Thread Execution Model..... 5

2.5 Memory Access Pattern..... 6

3. Loop Unrolling Implementation and Justification..... 6

3.1 Motivation for Loop Unrolling..... 6

3.2 Implementation Details 6

3.3 Code-Level Strategy 7

3.4 Performance Benefits 8

3.5 Limitations 8

4. Experimental Setup 8

4.1 Hardware Environment 8

4.2 Software Setup 9

4.3 Experiment Parameters..... 9

5. Analysis of Execution Time and RAM Usage..... 9

5.1 Execution Time vs Matrix Size 12

5.2 Effect of Thread Count..... 12

5.3 Float vs Double Comparison..... 12

5.4 RAM Usage Analysis 12

5.5 Limitations for Large Matrix Sizes 13

6. Impact of Loop Unrolling 13

6.1 Execution Time Comparison..... 13

6.2 Effect Across Matrix Sizes..... 13

6.3 RAM Usage Impact..... 13

6.4 Different Unrolling Factors 13

7. Known Issues and Limitations..... 14

7.1 Performance Limitations 14

7.2 Scalability Issues 14

7.3 Numerical Precision 14

7.4 System Constraints 14

8. Manual..... 15

8.1 Compilation..... 15

8.2 Execution..... 15

8.3 Execution Example Commands 15

8.4 Output Explanation..... 15

9. Conclusion 16

0. Introduction

This project focuses on the design and implementation of a parallel algorithm for the multiplication of three matrices using the pthreads library in C. The primary objective is to analyse the performance differences between sequential and parallel computation, particularly in terms of execution time and memory usage.

In addition, the project explores performance optimisation through the application of loop unrolling techniques. By comparing implementations with and without loop unrolling, the study evaluates how low-level optimisations can improve computational efficiency.

The performance of the program is assessed using two key factors: execution time and RAM consumption. Experiments are conducted under varying matrix sizes, two data types (float and double), and thread counts to provide a comprehensive evaluation of scalability and efficiency.

1. Problem Definition and Requirements

1.1 Problem Description

Suppose that m, k, l and n are positive integers. Four matrices A, B, C and D are defined with the following dimensions:

- *Matrix A* $\in R^{m \times k}$
- *Matrix B* $\in R^{k \times l}$
- *Matrix C* $\in R^{l \times n}$
- *Matrix D* $\in R^{m \times n}$

The objective of this program is to compute the matrix product:

- $D = A \times B \times C$

To improve computational efficiency, the program dynamically determines the optimal multiplication order between:

- $D = (A \times B) \times C$ and
- $D = A \times (B \times C)$

The selection is based on minimising the total number of arithmetic operations required for matrix multiplication.

1.2 Key Requirements

The program must satisfy the following requirements:

- Accept six command-line arguments: m, k, l, n , data type (float or double), and number of threads
- Support both float and double data types
- Use dynamic memory allocation for all matrices
- Implement both sequential and parallel (pthread-based) versions
- Use 1D row-wise partitioning for workload distribution
- Ensure correctness through result verification
- Measure execution time and compute speedup
- Measure RAM usage during execution
- Disable compiler optimisations (using `-O0`)

1.3 Constraints

- The number of threads must be between 1 and 64
- Only 1D row partitioning is allowed
- Task 1 must be implemented without loop unrolling
- Task 2 must implement loop unrolling with exactly two unrolled loops

2. Parallel Algorithm Design and Workload Distribution

2.1 Algorithm Overview

When the program is executed, it first reads all input parameters, including the matrix dimensions m, k, l, n , data type, and number of threads. Based on these inputs, the program determines the most efficient order for performing the multiplication of the three matrices.

Since matrix multiplication is associative, the computation of $D = A \times B \times C$ can be performed in two possible ways:

- $D = (A \times B) \times C$ and
- $D = A \times (B \times C)$

Although both approaches produce the same result, they may differ significantly in computational cost because we will calculate large size of matrices. Therefore, the program evaluates both options and selects the one that minimises the total number of arithmetic operations.

2.2 Cost-Based Order Selection

The efficiency of each multiplication order is determined by estimating the number of scalar multiplications required.

The cost of computing $(A \times B) \times C$ is:

- First multiply $A(m \times k)$ and $B(k \times l)$: cost = $m \times k \times l$
- Then multiply the result $m \times l$ with $C(l \times n)$: cost = $m \times l \times n$
- Total cost: cost1 = $m \times k \times l + m \times l \times n$

The cost of computing $A \times (B \times C)$ is:

- First multiply $B(k \times l)$ and $C(l \times n)$: cost = $k \times l \times n$
- Then multiply $A(m \times k)$ with the result $(k \times n)$: cost = $m \times k \times n$
- Total cost: cost2 = $k \times l \times n + m \times k \times n$

The program compares these two costs and selects the more efficient computation order using:

```
long cost1 = (long)m*k*l + (long)m*l*n;
long cost2 = (long)k*l*n + (long)m*k*n;

int mode = (cost1 <= cost2) ? 0 : 1;
```

If `cost1` is smaller, the program computes $(A \times B) \times C$; otherwise, it computes $A \times (B \times C)$. Choosing the optimal multiplication order reduces the total number of arithmetic operations, which directly decreases execution time. Since matrix multiplication has a time complexity of $O(n^3)$ even small reductions in the number of operations can lead to significant performance improvements, especially for large matrices.

Additionally, a more efficient order can improve cache utilisation and reduce memory access overhead, further enhancing overall performance.

2.3 Workload Distribution Strategy

A one-dimensional (1D) row-wise partitioning strategy is used to distribute the workload among multiple threads. In this approach, the rows of the result matrix are divided into contiguous blocks, and each thread is assigned a distinct subset of rows to compute.

The partitioning is implemented by calculating the starting and ending row indices for each thread as follows:

```
td[i].start_row = i * rows / actual;
td[i].end_row   = (i + 1) * rows / actual;
```

This method ensures that:

- The workload is evenly distributed across all threads
- There **is** no overlap between threads
- Each thread operates independently on its assigned rows

Each thread is provided with the necessary matrix data and dimensions, including references to matrices A, B, and the result matrix, along with parameters such as the number of rows, columns, and shared dimension.

During execution, each thread computes only its assigned rows of the result matrix. The nested loop structure follows an i-k-j ordering, as shown below:

```
for (int i = d->start_row; i < d->end_row; i++) {
    for (int k = 0; k < d->common; k++) {
        float a_val = A[i * d->common + k]; // reuse value
        for (int j = 0; j < d->cols; j++) {
            C[i * d->cols + j] += a_val * B[k * d->cols + j];
        }
    }
}
```

This loop ordering improves performance by:

- Accessing matrix A in a row-wise manner
- Reusing the value *a_val* to reduce repeated memory access
- Accessing matrix B sequentially across rows, improving cache locality

Because each thread writes only to its own assigned rows of the result matrix, race conditions are avoided, and no synchronisation mechanisms (such as locks) are required.

2.4 Thread Execution Model

Threads are created after initialising all matrices and thread-specific data structures. Each thread is assigned a unique subset of rows of the result matrix and executes the worker function independently:

```
pthread_create(&tids[i], NULL, worker, &td[i]);
```

The main thread synchronises execution by waiting for all threads to complete using:

```
pthread_join(tids[i], NULL);
```

This ensures that all computations are finished before proceeding.

Since each thread writes only to its assigned rows, no race conditions occur, and therefore no locking mechanisms are required. This design minimises synchronisation overhead and improves overall efficiency.

2.5 Memory Access Pattern

The implementation adopts a cache-friendly memory access pattern to improve performance. Specifically, the loop order follows an i-k-j structure instead of the human-friendly i-j-k ordering.

In this approach:

- Matrix A is accessed row-wise, which aligns with its memory layout
- Matrix B is accessed sequentially across rows within the innermost loop
- Matrix C is updated row-wise

To reduce repeated memory access, a temporary variable (`a_val`) is used to store the value of $A[i][k]$. This avoids reloading the same value multiple times within the inner loop, thereby improving cache utilisation.

```
for (int i = d->start_row; i < d->end_row; i++) {
    for (int k = 0; k < d->common; k++) {
        float a_val = A[i * d->common + k]; // reuse value
        for (int j = 0; j < d->cols; j++) {
            C[i * d->cols + j] += a_val * B[k * d->cols + j];
        }
    }
}
```

This design improves cache locality by ensuring sequential memory access and reducing cache misses, which leads to better overall performance.

3. Loop Unrolling Implementation and Justification

3.1 Motivation for Loop Unrolling

Loop unrolling is applied as a low-level optimisation technique to improve computational performance. It involves explicitly expanding loop iterations in code, thereby reducing reliance on loop control mechanisms.

By unrolling loops, the program reduces overhead associated with:

- loop index updates(increment, decrement)
- conditional checks
- branch instructions

In addition, loop unrolling increases instruction-level parallelism (ILP) by allowing multiple independent operations to be executed simultaneously by the CPU.

As a result, loop unrolling can significantly improve execution efficiency, particularly for compute-intensive operations such as matrix multiplication.

3.2 Implementation Details

Loop unrolling is implemented with an unrolling factor of 4, as initially required. Two levels of nested loops are unrolled:

- the k-loop (outer computation loop)
- the j-loop (inner accumulation loop)

This allows multiple elements to be processed simultaneously within a single iteration, significantly reducing loop overhead.

3.3 Code-Level Strategy

The implementation applies several optimisation techniques:

- **Multiple accumulators (a0–a3):**
Four consecutive elements of matrix *A* are loaded into temporary variables to enable parallel computation.
- **Pointer arithmetic:**
Row pointers for matrix *B* (e.g., *b0*, *b1*, *b2*, *b3*) are used to reduce index computation overhead and improve memory access efficiency.
- **Two-level loop unrolling:**
 - The *k*-loop is incremented by 4 (*k* += 4), processing four values per iteration
 - The *j*-loop is also unrolled by 4, allowing 16 multiplications per iteration

```
for (int k = 0; k < k_limit; k += 4) {
    float a0 = A[row_i_A + k];
    float a1 = A[row_i_A + k + 1];
    float a2 = A[row_i_A + k + 2];
    float a3 = A[row_i_A + k + 3];

    float *b0 = &B[k * c];
    float *b1 = &B[(k + 1) * c];
    float *b2 = &B[(k + 2) * c];
    float *b3 = &B[(k + 3) * c];

    int j = 0;
    for (; j < j_limit; j += 4) {
        int idx = row_i_C + j;

        C[idx]      += a0*b0[j]   + a1*b1[j]   + a2*b2[j]   + a3*b3[j];
        C[idx + 1] += a0*b0[j+1] + a1*b1[j+1] + a2*b2[j+1] + a3*b3[j+1];
        C[idx + 2] += a0*b0[j+2] + a1*b1[j+2] + a2*b2[j+2] + a3*b3[j+2];
        C[idx + 3] += a0*b0[j+3] + a1*b1[j+3] + a2*b2[j+3] + a3*b3[j+3];
    }

    // j remainder
    for (; j < c; j++) {
        C[row_i_C + j] += a0*b0[j] + a1*b1[j] + a2*b2[j] + a3*b3[j];
    }
}
```

- **Remainder handling:**
Additional loops are used to handle cases where the matrix dimensions are not divisible by 4:

```
// k remainder
for (int k = k_limit; k < common; k++) {
    float a = A[row_i_A + k];
    float *bk = &B[k * c];
    for (int j = 0; j < c; j++) {
        C[row_i_C + j] += a * bk[j];
    }
}
```

This design reduces loop overhead, improves instruction-level parallelism, and enhances cache efficiency, leading to better overall performance.

3.4 Performance Benefits

Loop unrolling improves performance by reducing the total number of loop iterations and minimising branching overhead from loop control instructions. This enables more efficient use of CPU resources and increases instruction-level parallelism, resulting in better overall utilisation of the processor.

3.5 Limitations

Despite its benefits, loop unrolling introduces increased register pressure, as more temporary variables are required. Additionally, performance gains exhibit diminishing returns as the unrolling factor increases, due to hardware limitations such as cache size and memory bandwidth.

4. Experimental Setup

4.1 Hardware Environment

```
jin@vlan-2633-10-19-21-160 ~ % system_profiler SPHardwareDataType
```

Hardware:

Hardware Overview:

```
Model Name: MacBook Air
Model Identifier: MacBookAir10,1
Model Number: MGN63KH/A
Chip: Apple M1
Total Number of Cores: 8 (4 Performance and 4 Efficiency)
Memory: 8 GB
System Firmware Version: 13822.81.10
OS Loader Version: 13822.81.10
Serial Number (system): FVFHCC4WQ6L4
Hardware UUID: 4CCBA067-5CD8-5D0C-AA70-8C9654D15F66
Provisioning UDID: 00008103-0000693001FB001E
Activation Lock Status: Disabled
```

```
jin@vlan-2633-10-19-21-160 ~ % sw_vers
```

```
ProductName:      macOS
ProductVersion:   26.3
BuildVersion:     25D125
```

```
[jin@vlan-2633-10-19-21-160 ~ % gcc --version
```

```
Apple clang version 17.0.0 (clang-1700.3.19.1)
Target: arm64-apple-darwin25.3.0
Thread model: posix
InstalledDir: /Library/Developer/CommandLineTools/usr/bin
```

All experiments were conducted on a MacBook Air equipped with an Apple M1 processor. The system has 8 CPU cores, consisting of 4 performance cores and 4 efficiency cores, and is equipped with 8 GB of RAM. The operating system used was macOS version 26.3, running on the Darwin kernel (Darwin Kernel Version 25.3.0) with arm64 architecture. The program was compiled using Apple clang version 17.0.0 (clang-1700.3.19.1), with compiler optimisations disabled using the -O0 flag as required.

4.2 Software Setup

The program was compiled using the following command with compiler optimisations disabled:

```
gcc -O0 -lpthread
```

Execution was performed via the command line, where matrix dimensions, data type, and thread count were provided as input arguments.

4.3 Experiment Parameters

The experiments were conducted under varying configurations to evaluate performance:

- Matrix sizes:
Sizes were increased exponentially from 2^8 to 2^{13} (i.e., 256 to 8192), subject to system limitations.
- Thread counts:
1, 2, 4, 8, 16, 32, 64
- Data types:
Comparison between float and double
- Implementations:
 - Task 1: without loop unrolling
 - Task 2: with loop unrolling (factor of 4)

5. Analysis of Execution Time and RAM Usage

To evaluate the performance of the implemented algorithms, a series of experiments were conducted using automated shell scripts using

```
for size in 256 512 1024 2048 4096 8192
do
  for threads in 1 2 4 8 16 32 64
  do
    for type in float double
    do
      echo "SIZE=$size THREADS=$threads TYPE=$type" >> task1_results.txt
      ./task1 $size $size $size $size $type $threads >> task1_results.txt
      echo "-----" >> task1_results.txt
    done
  done
done
```

```
for size in 256 512 1024 2048 4096 8192
do
  for threads in 1 2 4 8 16 32 64
  do
    for type in float double
    do
      echo "SIZE=$size THREADS=$threads TYPE=$type" >> task2_results.txt
      ./task2 $size $size $size $size $type $threads >> task2_results.txt
      echo "-----" >> task2_results.txt
    done
  done
done
```

These scripts systematically varied matrix sizes, thread counts, and data types, and executed both Task 1 (without loop unrolling) and Task 2 (with loop unrolling). The results, including execution time, speedup, and memory usage, were recorded, and organised into text files for analysis. These tables are the result.

task1 result

Size	Threads	Type	Parallel Time	Sequential Time	Speedup	Max RAM (MB)
256	1	float	0.068223	0.070164	1.028450	2.69
256	1	double	0.067020	0.067181	1.002401	4.19
256	2	float	0.034794	0.077822	2.236650	2.69
256	2	double	0.042669	0.065405	1.532847	4.19
256	4	float	0.019163	0.063582	3.317972	2.69
256	4	double	0.022695	0.071109	3.133239	4.19
256	8	float	0.021902	0.064950	2.965471	2.69
256	8	double	0.029071	0.076491	2.631174	4.19
256	16	float	0.017850	0.070797	3.966181	2.70
256	16	double	0.017623	0.065345	3.707952	4.19
256	32	float	0.015359	0.063490	4.133683	2.97
256	32	double	0.016537	0.066499	4.021179	4.22
256	64	float	0.016114	0.066628	4.134789	3.47
256	64	double	0.016248	0.065076	4.005180	4.72
512	1	float	0.520426	0.510875	0.981648	7.20
512	1	double	0.765693	0.703257	0.918458	12.84
512	2	float	0.270614	0.523195	1.933363	7.20
512	2	double	0.271283	0.526736	1.941646	13.22
512	4	float	0.149626	0.631192	4.218464	7.20
512	4	double	0.136928	0.528335	3.858484	13.22
512	8	float	0.126318	0.504757	3.995923	7.20
512	8	double	0.130660	0.509889	3.902410	13.22
512	16	float	0.118236	0.507241	4.290070	7.20
512	16	double	0.117193	0.721185	6.153826	13.22
512	32	float	0.113713	0.504440	4.436078	7.20
512	32	double	0.114699	0.514970	4.489747	13.22
512	64	float	0.112101	0.514031	4.585424	7.25
512	64	double	0.113698	0.509482	4.481012	13.22
1024	1	float	4.156593	4.041864	0.972398	24.88
1024	1	double	4.155287	4.066336	0.978593	49.28
1024	2	float	2.040508	4.037721	1.978782	25.28
1024	2	double	2.073182	4.085580	1.970681	49.28
1024	4	float	1.095867	4.100964	3.742210	25.28
1024	4	double	1.159289	4.075602	3.515605	49.28
1024	8	float	0.917837	4.015298	4.374739	25.28
1024	8	double	0.989994	4.292305	4.335688	49.28
1024	16	float	0.993216	4.290149	4.319453	25.28
1024	16	double	0.968695	4.095053	4.227391	49.28
1024	32	float	0.958705	4.036517	4.210385	25.28
1024	32	double	0.980968	4.138848	4.219147	49.28
1024	64	float	0.948655	4.049657	4.268841	25.28
1024	64	double	1.023850	4.101195	4.005659	49.28
2048	1	float	32.268263	32.724380	1.014135	81.30
2048	1	double	32.898346	37.154982	1.129388	161.30
2048	2	float	17.183586	32.812927	1.909551	86.25

task2 result

Size	Threads	Type	Parallel Time	Sequential Time	Speedup	Max RAM (MB)
256	1	float	0.021214	0.067950	3.203073	2.70
256	1	double	0.021381	0.064802	3.030833	4.20
256	2	float	0.011275	0.063493	5.631283	2.70
256	2	double	0.012289	0.064795	5.272583	4.20
256	4	float	0.005866	0.063785	10.873598	2.70
256	4	double	0.006263	0.064113	10.236743	4.20
256	8	float	0.005044	0.063797	12.648138	2.70
256	8	double	0.005385	0.063980	11.881342	4.20
256	16	float	0.004908	0.064805	13.203731	2.75
256	16	double	0.005069	0.064950	12.813132	4.20
256	32	float	0.005832	0.063427	10.875761	2.98
256	32	double	0.005182	0.063998	12.349988	4.25
256	64	float	0.005269	0.063284	12.010498	3.45
256	64	double	0.005791	0.066344	11.456503	4.70
512	1	float	0.174456	0.581960	3.335859	7.22
512	1	double	0.170001	0.514925	3.028952	13.23
512	2	float	0.087634	0.509022	5.808509	7.22
512	2	double	0.089744	0.509441	5.676611	13.23
512	4	float	0.045249	0.506741	11.198948	7.22
512	4	double	0.046557	0.513675	11.033266	13.23
512	8	float	0.039489	0.509355	12.898641	7.22
512	8	double	0.040811	0.509660	12.488281	13.23
512	16	float	0.039719	0.506635	12.755447	7.22
512	16	double	0.040237	0.510746	12.693457	13.23
512	32	float	0.036734	0.504489	13.733531	7.22
512	32	double	0.040341	0.514458	12.752769	13.23
512	64	float	0.037743	0.507773	13.453492	7.27
512	64	double	0.036664	0.509337	13.892021	13.23
1024	1	float	1.322160	4.255256	3.218412	25.30
1024	1	double	1.415365	4.141371	2.926009	48.91
1024	2	float	0.679001	4.030371	5.935736	25.30
1024	2	double	0.724400	4.090894	5.647286	49.30
1024	4	float	0.355221	4.057835	11.423409	25.30
1024	4	double	0.382085	4.217796	11.038892	49.30
1024	8	float	0.328251	4.046442	12.327284	25.30
1024	8	double	0.323447	4.117424	12.729826	49.28
1024	16	float	0.295953	4.059492	13.716677	25.30
1024	16	double	0.312917	4.120062	13.166629	49.30
1024	32	float	0.300228	4.135837	13.775659	25.30
1024	32	double	0.306384	4.340937	14.168284	49.30
1024	64	float	0.281201	4.042129	14.374525	25.28
1024	64	double	0.315103	4.322913	13.719046	49.30
2048	1	float	10.968761	32.972831	3.006067	96.84
2048	1	double	11.794167	34.526249	2.927400	185.19
2048	2	float	5.855936	32.267496	5.510220	97.30
2048	2	double	6.456868	32.686988	5.062360	192.84
2048	4	float	3.202309	32.577383	10.173091	97.30
2048	4	double	3.803673	33.090328	8.699572	192.81
2048	8	float	2.618038	32.611251	12.456370	97.30
2048	8	double	2.947979	33.164854	11.250031	192.84
2048	16	float	2.644298	32.788426	12.399672	85.59
2048	16	double	4.002151	35.782107	8.940719	172.83

5.1 Execution Time vs Matrix Size

The experimental results show that execution time increases significantly as matrix size grows. For example, in Task 1 (float), the parallel execution time increases from approximately 0.068 s (256) to 4.16 s (1024) and further to 32.27 s (2048). This represents an approximate $60\times$ increase from 256 to 1024 and nearly $8\times$ from 1024 to 2048, which is consistent with the theoretical time complexity of matrix multiplication, $O(n^3)$, due to the use of triple nested loops.

A similar pattern is observed in Task 2, although execution times are consistently lower due to loop unrolling. The cubic growth becomes more pronounced for larger matrices, where computational cost dominates execution and memory access latency becomes increasingly significant.

5.2 Effect of Thread Count

Increasing the number of threads improves performance up to a certain point. In Task 1 (1024, float), speedup increases from approximately $1.97\times$ (2 threads) to $4.37\times$ (8 threads), but then stabilises around $4.2\text{--}4.3\times$ for 16–64 threads, indicating diminishing returns.

This behaviour is expected due to hardware limitations. The Apple M1 processor has only 4 performance cores, and once the number of threads exceeds the number of physical cores, additional threads introduce overhead such as context switching and scheduling. As a result, the parallel efficiency decreases, and the speedup saturates.

This observation is consistent with Amdahl's Law, where the serial portion of the program and system overhead limit the achievable speedup regardless of additional threads.

5.3 Float vs Double Comparison

The results indicate that double precision computations are generally slower and require more memory compared to float. For instance, at size 1024, RAM usage increases from approximately 25 MB (float) to 49 MB (double), which is nearly a $2\times$ increase, consistent with the difference in data size (4 bytes vs 8 bytes).

Execution time differences are relatively moderate. For example, at size 1024 with 8 threads, float takes approximately 0.917 s, while double takes 0.989 s, representing only about an 8% increase. This suggests that computation is still the dominant factor, although increased memory footprint reduces cache efficiency and slightly impacts performance.

5.4 RAM Usage Analysis

Memory usage increases proportionally with matrix size and follows a quadratic growth pattern $O(n^2)$, since storage depends on the number of matrix elements.

For example:

- 256 $\rightarrow$ $\sim$ 2–4 MB
- 512 $\rightarrow$ $\sim$ 7–13 MB
- 1024 $\rightarrow$ $\sim$ 25–49 MB
- 2048 $\rightarrow$ $\sim$ 80–160 MB

This represents roughly a $4\times$ increase in memory usage when matrix size doubles, which aligns with theoretical expectations.

RAM usage remains largely independent of thread count, as threads share the same memory space. The increase is primarily due to the storage requirements of matrices A , B , C , and intermediate results. However, for larger matrices, increased memory usage contributes to cache misses and memory bandwidth pressure, which negatively impacts performance.

5.5 Limitations for Large Matrix Sizes

For larger matrix sizes (2048, 4096 and 8192), experiments could not be fully completed due to system limitations. Execution time exceeded 5 minutes per run, and memory consumption approached system limits, leading to instability.

This limitation is primarily due to:

- high computational complexity $O(n^3)$
- increased memory consumption
- limited hardware resources (8 GB RAM and cache size)

Despite these constraints, the observed trends for smaller matrix sizes are consistent and sufficient to validate the performance behaviour of the algorithm.

6. Impact of Loop Unrolling

6.1 Execution Time Comparison

Loop unrolling significantly improves performance across all matrix sizes. For example, at size 1024 (float, 64 threads), speedup increases from approximately $4.27\times$ (Task 1) to $14.37\times$ (Task 2), representing more than a $3\times$ improvement.

This improvement is attributed to:

- reduced loop control overhead (fewer increments and condition checks)
- fewer branch instructions
- increased instruction-level parallelism (ILP)

As a result, the CPU can execute more arithmetic operations per cycle, improving overall throughput.

6.2 Effect Across Matrix Sizes

The benefits of loop unrolling are more substantial for larger matrix sizes. As the problem size increases, the proportion of time spent on arithmetic operations grows, allowing loop unrolling to have a greater impact.

For smaller matrices, performance gains are less pronounced because overhead (thread management and memory access) dominates execution time. In contrast, for larger matrices, computation dominates, making optimisation techniques such as loop unrolling more effective.

6.3 RAM Usage Impact

Loop unrolling has no significant effect on RAM usage, as it optimises computation rather than memory allocation. Memory consumption remains primarily dependent on matrix size.

6.4 Different Unrolling Factors

Additional experiments were conducted by modifying Task 2 to use different unrolling factors (1, 2, and 4). The results indicate that an unrolling factor of 4 provides the best performance.

However, performance improvements are not linear. Increasing the unrolling factor beyond a certain point leads to:

- increased register pressure
- reduced cache efficiency

- instruction pipeline limitations

These factors limit further performance gains.

```
jin@vlan-2633-10-19-21-160 assginment % ./task2_unroll 1024 2048 4096 1024 float 4 2
Unroll: 2 | SUCCESS
Speedup: 2.26x | Parallel: 10.5663s
RAM: 77.34 MB
jin@vlan-2633-10-19-21-160 assginment % ./task2_unroll 1024 2048 4096 1024 float 4 1
Unroll: 1 | SUCCESS
Speedup: 1.55x | Parallel: 13.6158s
RAM: 80.72 MB
jin@vlan-2633-10-19-21-160 assginment % ./task2_unroll 1024 2048 4096 1024 float 4 4
Unroll: 4 | SUCCESS
Speedup: 2.55x | Parallel: 8.3440s
RAM: 80.64 MB
```

6.5 Key Insight

The results demonstrate that selecting an appropriate unrolling factor is critical. While higher factors can improve performance, they also increase code complexity and hardware pressure.

An unrolling factor of 4 achieves a good balance between:

- reducing loop overhead
- maintaining manageable register usage
- improving cache utilisation

Therefore, it provides the most effective optimisation in this implementation.

7. Known Issues and Limitations

7.1 Performance Limitations

The performance of the implementation is constrained by memory bandwidth and cache efficiency. As matrix sizes increase, frequent memory access leads to cache misses and limits the effectiveness of parallel computation.

7.2 Scalability Issues

Although increasing the number of threads improves performance initially, the speedup is limited beyond a certain point. This is due to thread management overhead and the finite number of physical CPU cores, resulting in diminishing returns for higher thread counts.

7.3 Numerical Precision

The use of floating-point arithmetic may introduce rounding errors, particularly for large matrix sizes or when using single precision (float). While these errors are generally small, they may affect exact numerical accuracy.

7.4 System Constraints

The system has practical limitations in handling very large matrix sizes. Experiments with larger inputs (e.g., 2048 and above) were not feasible due to excessive execution time and memory constraints on the available hardware.

8. Manual

8.1 Compilation

The programs are compiled using the following commands with compiler optimisations disabled:

```
gcc task1.c -o task1 -lpthread -O0
gcc task2.c -o task2 -lpthread -O0
```

8.2 Execution

The programs are executed via the command line with the following format:

```
./task1 m k l n float threads
./task2 m k l n float threads
```

where m, k, l, n define matrix dimensions, the data type is either `float` or `double`, and the number of threads ranges from 1 to 64.

8.3 Execution Example Commands

```
./task1 1024 2048 4096 1024 float 64
./task2 1024 2048 4096 1024 float 64
```

8.4 Output Explanation

The program outputs the following information:

- Verification: Indicates correctness of results (SUCCESS or FAIL)
- Parallel Time: Execution time of the parallel implementation (seconds)
- Sequential Time: Execution time of the sequential implementation (seconds)
- Speedup: Ratio of sequential to parallel execution time
- RAM Usage: Maximum memory consumption (MB)

The expected outcome is a successful verification with improved performance in the parallel implementation, particularly when loop unrolling is applied.

```
jin@vlan-2633-10-19-21-160 assginment % gcc task1-google.c -o task1 -lpthread -O0
jin@vlan-2633-10-19-21-160 assginment % ./task1 1024 2048 4096 1024 float 64
Verification: SUCCESS
Parallel Time: 4.958261
Sequential Time: 21.587894
Speedup: 4.353924
Max RAM: 70.28 MB
```

```
jin@vlan-2633-10-19-21-160 assginment % gcc task2.c -o task2 -lpthread -O0
jin@vlan-2633-10-19-21-160 assginment % ./task2 1024 2048 4096 1024 float 64
Verification: SUCCESS
Parallel Time: 1.730193
Sequential Time: 20.963282
Speedup: 12.116151
Max RAM: 73.30 MB
```

9. Conclusion

This study demonstrates that parallelisation using pthreads significantly improves the performance of matrix multiplication compared to the sequential implementation. The results show that execution time decreases and speedup increases as the number of threads grows, although gains are limited by hardware constraints such as the number of CPU cores and memory bandwidth.

The application of loop unrolling further enhances performance by reducing loop overhead and increasing instruction-level parallelism. Experimental results indicate that loop unrolling provides substantial additional speedup, particularly for larger matrix sizes, with an unrolling factor of 4 achieving the best balance between performance and complexity.

Overall, the key findings confirm that both parallelisation and low-level optimisation techniques are essential for improving computational efficiency in matrix operations. However, their effectiveness is influenced by system architecture, memory hierarchy, and implementation design, highlighting the importance of selecting appropriate optimisation strategies.